* CONTEXT WINDOWS

Context windows: the cheat sheet

A context window is the temporary workspace an AI reads from for one response. This sheet shows what goes into it, what happens when it fills up, and how to keep an important rule in front of the model during a long chat.

The package in one view

```
YOU          message + files + your latest question
              |
              v
THE APP      builds a package of tokens for this one answer:
              app instructions + saved preferences or summaries (if the product has them)
              + recent messages + uploaded files + your latest question
              |   the package must fit the context window
              v
THE MODEL    reads only that package, writes one answer
              (the answer counts toward the window too)
              |
              v
THE APP      builds a new package for the next answer
```

A token is a word, part of a word, punctuation or code. For English text, one token is roughly 4 characters or three-quarters of a word, so 100 tokens is about 75 words. Tokenizers differ by model: the sentence `Use TypeScript. Do not change the payment tests.` is 11 tokens in the GPT-2 encoding, 10 in `cl100k_base` and 11 in `o200k_base`.

When the window fills up

WHAT THE PRODUCT DOES	WHAT STAYS	WHAT CAN BE LOST	WHAT YOU CAN DO
Drops the oldest messages	The newest messages	The start of the chat, where rules usually live	Repeat the rule in a recent message
Replaces old messages with a summary	The gist	The exact wording of a rule	Ask for rules to be kept word for word, or restate them
Keeps instructions in a separate slot	The pinned text, every time	Little, but pinned text still takes up window space	Put hard rules in the persistent instructions slot
Searches the earlier chat and adds parts back	The parts that match your latest question	Anything the search did not match	Mention the rule's topic in your question

Measured on a stand-in chat of 1,140 tokens in GPT-2's real 1,024-token window: dropping the oldest messages removed the rule first, the summary squeezed 1,046 tokens into 38 and lost the exact words, pinning kept the rule, and a word-overlap search brought it back. These are minimal strategies for teaching, not any product's algorithm.

Window, memory and training are different things

THING	WHERE IT LIVES	HOW LONG IT LASTS
Context window	The package built for one response	This one response
Saved memory or custom instructions	Product software around the model, added to the package	Until you delete it
Training	The model's weights	Set in training, not changed by your chat

LONG-CHAT HABITS

- Put the rule in the persistent instructions slot if the product has one.
- Restate a rule in your latest message before a big change.
- Paste only the part of a file the question needs.
- Ask the model to quote the relevant part first.
- When a chat gets long, start a new one with a short brief.

Remember: A context window is not everything an AI knows or remembers forever. It is the temporary workspace that decides what the AI can use right now.

REFERENCE

Advanced reference

Each section gives you something to run, check or decide. The code runs on Python with `pip install tiktoken`; the first run downloads the encoding files, so it needs a network connection.

1. Count tokens before you trust a window

Characters and words are only an estimate. Count with the tokenizer your model uses, because the same text is a different number of tokens in different encodings.

```
import tiktoken

rule = "Use TypeScript. Do not change the payment tests."
for name in ("gpt2", "cl100k_base", "o200k_base"):
    enc = tiktoken.get_encoding(name)
    pieces = [enc.decode([i]) for i in enc.encode(rule)]
    print(f"{name:<12} {len(pieces):>2} tokens  {pieces}")
print("characters:", len(rule), "-> about", round(len(rule) / 4), "tokens by the 4-characters rule")
```

Output:

```
gpt2         11 tokens  ['Use', ' Type', 'Script', '.', ' Do', ' not', ' change', ' the', ' payment', ' tests', '.']
cl100k_base  10 tokens  ['Use', ' TypeScript', '.', ' Do', ' not', ' change', ' the', ' payment', ' tests', '.']
o200k_base   11 tokens  ['Use', ' Type', 'Script', '.', ' Do', ' not', ' change', ' the', ' payment', ' tests', '.']
characters: 48 -> about 12 tokens by the 4-characters rule
```

The count includes your own message, the answer, uploaded files and any instructions the app adds, so leave room. A real assistant's window is far larger than GPT-2's 1,024 tokens, and it changes by model: check the documentation for the one you use.

2. Watch a rule fall out of a window

A toy chat of 157 tokens in a 130-token budget. Four minimal strategies decide what to keep. A fixed budget forces a choice, and two of the four lose the rule.

```
import re
import tiktoken

enc = tiktoken.get_encoding("gpt2")
count = lambda text: len(enc.encode(text))

chat = [
    ("app", "You are a coding assistant. Answer briefly and show code."),
    ("rule", "Use TypeScript. Do not change the payment tests."),
    ("docs", "POST /v1/charges creates a charge. Amounts are integers in cents. A 402 means the card was declined."),
    ("log", "TypeError: Cannot read properties of undefined (reading 'price') at calcTotal (checkout.ts:48)"),
    ("code", "export function calcTotal(cart: Cart): number { return cart.items.reduce((t, i) => t + i.price * i.qty, 0);
}"),
    ("q1", "Why is the total NaN for some carts? Some items arrive without a price, so guard the multiplication."),
    ("q2", "Should the amount be sent in cents? Yes, round to an integer number of cents before charging."),
    ("latest", "Fix this last bug."),
]
BUDGET = 130  # a toy window, so the chat does not fit
total = sum(count(text) for _, text in chat)
print(f"chat: {total} tokens, budget: {BUDGET}")

def newest_that_fit(items, room):
    kept = []
    for key, text in reversed(items):
        if count(text) <= room:
            kept.insert(0, key)
            room -= count(text)
        else:
            break
    return kept

app, rule, latest = chat[0], chat[1], chat[-1]
middle = chat[2:-1]
fixed = count(app[1]) + count(latest[1])

drop = [app[0]] + newest_that_fit(chat[1:-1], BUDGET - fixed) + [latest[0]]
summary = "Summary: Alex is fixing a checkout bug in a TypeScript app and asked two questions."
summarise = [app[0], "summary", chat[-3][0], chat[-2][0], latest[0]]
pin = [app[0], rule[0]] + newest_that_fit(middle, BUDGET - fixed - count(rule[1])) + [latest[0]]
words = lambda t: set(re.findall(r"[a-z0-9]+", t.lower()))
query = words(latest[1] + " payment tests TypeScript")
```

```
ranked = sorted(chat[1:-1], key=lambda m: -len(words(m[1]) & query) / len(words(m[1]) | query))
retrieve = [app[0]] + [key for key, _ in ranked[:2]] + [latest[0]]

for name, kept in [("drop oldest", drop), ("summarise", summarise), ("pin", pin), ("retrieve", retrieve)]:
    size = sum(count(summary if k == "summary" else dict(chat)[k]) for k in kept)
    print(f"{name:<12} {size:>3} tokens  rule kept: {'rule' in kept}")
```

Output:

```
chat: 157 tokens, budget: 130
drop oldest  121 tokens  rule kept: False
summarise     77 tokens  rule kept: False
pin          108 tokens  rule kept: True
retrieve      53 tokens  rule kept: True
```

Read the last column, not the sizes. Summarising lost the rule because the summary text never contains it. Retrieval kept the rule only because the query mentioned the same words.

3. Debug a forgotten rule

SYMPTOM	LIKELY CAUSE	FIX
The AI ignores a rule from early in a long chat	The rule was dropped or summarised away	Restate it in a new message and keep it in the persistent instructions
A rule works at first, then drifts as the chat grows	A bigger pile makes recall less reliable even when everything fits	Trim pasted material and restate the rule near your question
The AI does not remember last week's chat	A context window is not saved memory	Use the product's memory feature or paste a short brief
A long answer is cut off	The answer counts toward the window	Shorten the input or ask for a shorter answer
The API returns "prompt is too long"	The input alone is larger than the window	Trim, summarise or compact the history
Every turn is slower or costs more	The API is stateless, so the history is sent again each turn and billed as input	Compact the history or start a new thread

4. For builders: managing context in your own app

OPTION	WHAT IT DOES	USE IT WHEN
Count first	Measure tokens before you send, with a token counting API or a tokenizer such as `tiktoken`	You need to know whether the package fits
Compaction	Replaces older turns with a summary the model writes; recent turns can stay word for word, and you can write your own summarization prompt	A long conversation approaches the limit and the default summary drops something later turns need
Clear old tool results	Removes raw results of earlier tool calls by rule	An agent has called many tools and the results are no longer needed
Notes outside the window	The agent writes notes to storage and pulls them back later	A task spans more than one window
Load on demand	Keep lightweight identifiers such as file paths, and load the data when it is needed	The data is large and only some of it is relevant

For long documents, put them near the top of the prompt and your question and instructions at the end. Asking the model to quote the relevant parts first helps it focus on them.

5. Checklist before you paste the next file

- [ ] Does the model need the whole file, or only one part of it?
- [ ] Are the rules that matter written somewhere persistent?
- [ ] Is my latest message restating the rule that the next change could break?
- [ ] Has the chat grown long enough that a fresh chat with a short brief would be safer?
- [ ] If the product keeps memory, is that memory switched on and holding what I think it holds?

Remember: If the model cannot see a rule, it cannot follow it, and a pile that fits can still bury it.